

Projeto de Automação: Atualizador Inteligente de ERP

Orquestração em Nuvem, Agentes Locais e Segurança de
Dados

Documento de Arquitetura e Apresentação
Executiva

1. O Desafio e Nosso Objetivo

Atualmente, o processo de atualização de versões nos servidores dos clientes demanda intervenção manual. Isso gera dependência da equipe de suporte, consome tempo útil e abre margem para falhas humanas (como esquecer de realizar um backup antes de aplicar um script de banco).

O Objetivo: Construir um ecossistema autônomo. O sistema deve ser capaz de detectar novas versões, baixar pacotes de forma silenciosa, solicitar permissão ao usuário de forma amigável e, com um único clique, realizar todo o trabalho duro (backups, scripts e distribuição) de forma 100% segura, respeitando as particularidades do Firebird 2.5.

2. A Arquitetura Escolhida (Quem faz o quê?)

Para garantir estabilidade, não vamos sobrecarregar o ERP atual com tarefas de infraestrutura. Dividiremos o trabalho em três pilares, usando a melhor tecnologia para cada função:

1. A Mente (Nuvem / API Central)

Construída em tecnologias modernas para web (como Python ou Node.js). Ela atua como um "maestro". Quando um cliente pergunta se há novidades, ela verifica o CNPJ, entende quais módulos aquele cliente possui (ex: BNFE, B_Importa) e entrega apenas os arquivos corretos. Isso economiza banda e mantém sistemas exclusivos protegidos.

2. O Músculo (Agente Local em C# / .NET)

Um Serviço do Windows invisível que roda 24h por dia no servidor do cliente. Escolhemos **C# (.NET 8)** porque ele é feito exatamente para isso: roda processos pesados em segundo plano, faz downloads assíncronos sem travar a máquina e conversa perfeitamente com o Firebird via driver oficial nativo. Tudo isso em um único arquivo, sem precisar instalar nada no cliente.

3. O Rosto (Nosso ERP Delphi)

O `b_vendas` e os terminais continuam em Delphi. A única mudança é que eles passarão a ler a tabela de controle e avisar o usuário: *"Ei, a atualização está pronta, podemos aplicar?"*. Eles mantêm a interface amigável que o cliente já conhece.

3. O Passo a Passo da Atualização (A Mágica)

Abaixo detalhamos o ciclo de vida completo de uma atualização, desde o lançamento pela nossa equipe até chegar à tela do cliente.

Fase 1: O Preparo Invisível

- O **Agente C#** acorda e pergunta à API: *"Tem algo novo para mim?"*.
- Se houver, ele baixa os pacotes (ex: `.7z` ou `DelphiXE.rar`).
- Ele usa um extrator universal e deixa os novos `.exe` e scripts prontos em uma pasta temporária.
- No banco principal (`JUNIOR.fdb`), ele muda a flag de uma tabela (`SYS_ATUALIZACAO`) para `'PENDENTE'`.

Fase 2: A Decisão do Usuário

- De manhã, quando o administrador abre o ERP (`b_vendas`), o sistema lê a flag `'PENDENTE'`.
- Uma tela amigável aparece: *"Nova versão pronta para instalar. O sistema precisará ser reiniciado. Atualizar agora?"*.
- Ao clicar em "Sim", o ERP muda a flag para `'AUTORIZADO'` e se fecha sozinho. O poder de decisão fica com o cliente, evitando que o sistema caia no meio de uma venda.

Fase 3: A Execução Segura (Trabalho Pesado)

Assim que o Agente lê `'AUTORIZADO'`, ele assume o controle do servidor e executa uma sequência blindada contra falhas:

1. **Isolamento da Área:** Roda `gfix -shut force_0` para expulsar qualquer conexão travada e impedir novos logins. (Garante que o Firebird não dará erro de "Object in use").
2. **Seguro de Vida (Backup 1):** Roda o `gbak`. Se a energia cair durante a atualização, temos o banco de minutos atrás salvo.
3. **Execução de Scripts:** Chama o nosso `BScript.exe` de forma silenciosa para aplicar as mudanças de banco de dados.
4. **Seguro de Vida (Backup 2):** Roda outro `gbak`. Agora temos um backup com a estrutura já modernizada.

Fase 4: A Distribuição Limpa

- O Agente conecta no banco de distribuição (`BEXE.fdb`).
- Ele lê os executáveis que baixou na Fase 1, converte para arquivos binários (BLOBs) e injeta no banco de forma extremamente rápida.
- O banco é liberado (`gfix -online`). Quando os terminais da loja abrirem, eles vão ler o `BEXE.fdb` e se auto-atualizar normalmente.

4. Decisões Críticas de Engenharia

A. O Problema da Compactação (.7z vs WinRAR)

Temos pacotes no formato `.7z` e pacotes antigos com compressão `.rar/.zip`. Para não obrigar os programadores a escreverem decodificadores complexos, o Agente C# levará consigo o `7za.exe` (motor invisível do 7-zip). O C# apenas dirá: *"Extraia tudo que estiver na pasta"*, e o 7-zip lidará com as diferentes extensões universalmente e sem falhas.

B. A Paz de Espírito com o Firebird 2.5

Sabemos que o Firebird 2.5 não gosta que alterem sua estrutura (DDL) enquanto há pessoas conectadas. A arquitetura de derrubar as conexões via código (GFIX) antes de rodar os scripts foi desenhada especificamente para tratar essa limitação nativa do banco, trazendo 100% de estabilidade ao processo.